Task-7:- Requirement Analysis

1. Introduction

1.1 Purpose

The purpose of this document is to outline the functional and technical requirements for an automated monitoring solution that tracks file accumulation in temporary directories. The intended audience includes DevOps engineers, SREs, and stakeholders responsible for infrastructure health.

1.2 Scope

- **Included:** Development of a Python/Go-based collector, containerization via Docker, deployment on AWS Lambda and ECS Fargate, integration with Prometheus Pushgateway, and Grafana dashboard/alerting setup.
- **Excluded:** Automatic deletion of files (remediation), monitoring of non-temporary directories, or setting up the base Prometheus/Grafana infrastructure from scratch.

1.3 Definitions, Acronyms, and Abbreviations

- **Pushgateway:** A routing service for short-lived Prometheus jobs.
- **Fargate:** A serverless compute engine for containers on AWS.
- **EventBridge:** A serverless event bus used here as a cron-scheduler.
- **Metric Gauge:** A numeric representation of a value that can go up and down.

1.4 References

- Prometheus Pushgateway Documentation
- AWS Lambda Container Image Support Guide
- Grafana Alerting Documentation

1.5 Overview

This document transitions from project objectives and business goals into technical use cases, non-functional constraints, and risk mitigation strategies to ensure a robust monitoring pipeline.

---

2. Project Overview

2.1 Project Objectives

- Automate the counting of files in a specific system directory.
- Centralize metrics in a Grafana dashboard for real-time visibility.
- Enable proactive alerting to prevent disk-fill issues.

2.2 Business Goals

- **Reduce Downtime:** Prevent system crashes caused by "disk full" errors.
- **Operational Efficiency:** Remove the need for manual directory checks.
- **Standardization:** Use containerized patterns that work across different AWS compute services.

2.3 Background

Legacy systems often accumulate temporary files that are not properly cleaned up by applications. Without a push-based monitoring system, these ephemeral environments (like Lambda) are difficult to observe using traditional "pull" scraping.

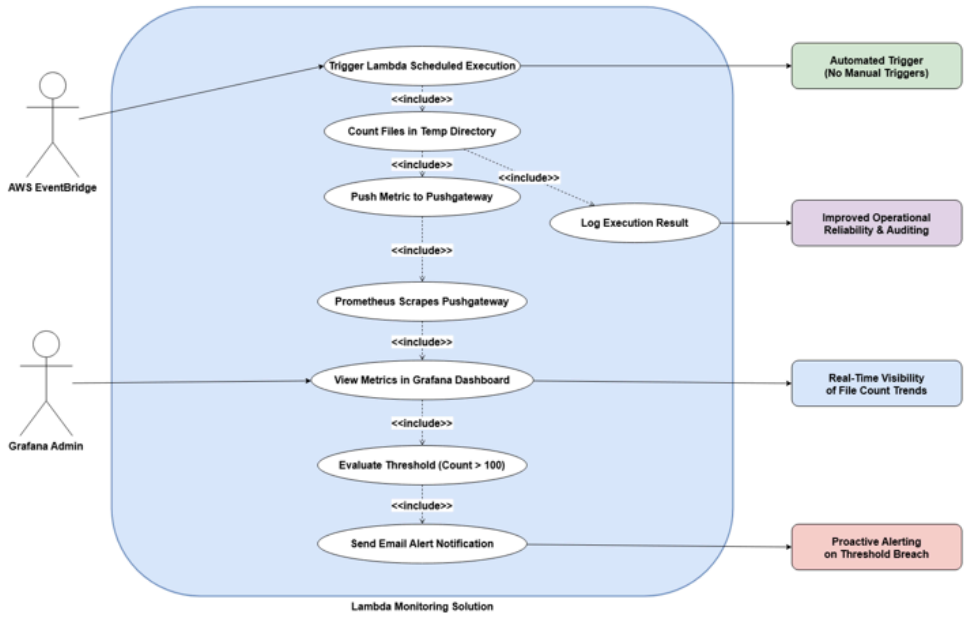
2.4 Constraints

- **Network:** The collector must have outbound access to the Pushgateway.
- **Environment:** Lambda is limited to the `/tmp` directory for local write/read tests.
- **Runtime:** The execution must complete within the Lambda timeout (default 3s, recommended 30s).

---

3. Functional Requirements

3.1 Use Case Diagram



3.2 Use Cases

3.2.1 Use Case 1: Metric Collection and Dispatch

- **Description:** The system counts files and pushes the count to the cloud.
- **Actors:** EventBridge (Trigger), Collector Script, Pushgateway.

- **Preconditions:** Collector is deployed with correct environment variables.
- **Postconditions:** Metric `temp_dir_file_count` is available in Prometheus.
- **Flow of Events:**
  a. EventBridge triggers the Lambda/ECS task.
  b. Collector reads `TEMP_DIR_PATH` from env vars.
  c. Collector performs a file count.
  d. Collector formats data to Prometheus text format.
  e. Collector sends an HTTP POST to `PUSHGATEWAY_URL`.
- **Alternative Flows:**
  - *Directory Missing:* Collector logs an error and pushes a "0" or "NaN" to indicate failure.

---

4. Non-Functional Requirements

- **4.1 Performance:** Metric collection and push must complete in < 2 seconds.
- **4.2 Security:** Use IAM roles with "Least Privilege" for Lambda execution and ECR access.
- **4.3 Usability:** Grafana dashboard should use color-coded thresholds (Green/Yellow/Red).
- **4.4 Reliability:** The system should handle Pushgateway timeouts gracefully without crashing the collector.
- **4.5 Compliance:** Ensure logs in CloudWatch do not contain Sensitive Personal Information (SPI).

---

5. Assumptions and Dependencies

- **5.1 Assumptions:** The Pushgateway is already configured to be scraped by a Prometheus instance.
- **5.2 Dependencies:** Availability of AWS ECR for storing the Docker images and a functional SMTP/Grafana contact point for emails.

---

6. Risk Analysis

- **6.1 Risk Identification:** High frequency of Lambda runs (every 1 min) might incur unexpected AWS costs.
- **6.2 Risk Mitigation:** Start with a 5-minute interval; use a lightweight base image (Alpine) to minimize data transfer costs.

---

7. Glossary

- **Collector:** The script responsible for the logic of counting and pushing.
- **Job/Instance Labels:** Metadata used by Prometheus to identify the source of a metric.

---

8. Appendices

- **Environment Variable Schema:**
  - `TEMP_DIR_PATH`: Path to monitor.
  - `PUSHGATEWAY_URL`: Endpoint (e.g., `http://pushgateway.internal:9091`).
  - `METRIC_NAME`: Default `temp_dir_file_count`.

---

9. Customer Acknowledgement

Name: _____

Title: _____

Signature: _____

Date: _____